Herald

Herald — Issues

Field	Value
Revision	15
Created	2026-05-20
Last modified	2026-05-22
Status	active
Status summary	**r15 captures Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure; tag v0.4.0 deferred to T13b post live evidence). HRD-100 atomic Issues→Fixed in this revision.** New Cobra subcommand `pherald listen` runs the closed-loop runtime: Telegram `getUpdates` long-poll (`OnText`/`OnPhoto`/`OnDocument`/`OnVoice`) + bot self-filter (`bot.Me.Username` anti-echo guard, fails Subscribe if `getMe` returned no username) + sha256 content-addressed attachment download into `~/.herald/inbox/<sha256>.<ext>` + Claude Code dispatch with **Opus pinned in argv** (`claude --model claude-opus-4-7 --resume <UUID> --print <envelope>`) + envelope pre-text (verbatim operator wording "We have received new message from our communication channel . ...") preceding the `<<<HERALD-DISPATCH-v1>>>` structured block + `<<<HERALD-REPLY>>>` parser with `action` routing (`reply` / `issue.open` / `event.emit`) + `tgram.Adapter.SendReply` wiring `reply_to_message_id` from the inbound `message_id`. T10a `--qa-out-dir` flag journals every inbound/outbound event as JSONL per §107.x. 12 commits (T1=`ad87d7f`..T12=`96c7c6b`). 8 new e2e invariants E63-E70 added (currently SKIP with documented reasons — converting to PASS at T10b when operator runs the live closed-loop and commits `docs/qa/HRD-100/`). Wave 6 mutation gate `tests/test_wave6_mutation_meta.sh` lands 3 paired hermetic mutations. Tag **v0.4.0 deferred to T13b** (operator-supplied live evidence per §107.x). Prior r14 captured Wave 4b TOON `application/toon` content negotiation (tag v0.3.0). Prior r13 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3). r12 closed HRD-016 (Wave 3b pherald Runner live). r11 closed HRD-011 (Telegram live, message_id=5).
Issues	HRD-008, HRD-015, HRD-018..HRD-027, HRD-029..HRD-056, HRD-081, HRD-085..HRD-090
Issues summary	47 open / in-progress workable items. r15: HRD-100 closed atomically (Wave 6 pherald inbound runtime — code-doc closure; live evidence operator-pending → T10b). r14: substrate upgrade only (Wave 4b TOON; no HRDs migrated). r12:

Field	Value
	HRD-016 closed atomically (Wave 3b pherald Runner live + e2e E37-E42 PASS or honest SKIP-with-reason when PG :24100 absent). r11: HRD-011 (Telegram live) closed atomically with live message_id=5 evidence. r10 doc-cleanup renumbered HRD-093 → HRD-099. r9: HRD-028 + HRD-098 closed (Wave 3a). Carry-over to Wave 3c: HRD-024 (iherald paging), HRD-033 (destructive-guard body), HRD-018..027 flavor-specific bindings, plus the rest of the §43 command catalogue.
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-028, HRD-080, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099, HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114
Fixed summary	see `Fixed.md`.
Continuation	see `CONTINUATION.md`.

Table of contents

- Open
- In progress
- Blocked
- Conventions

Open

Per Universal §11.4.74 every new row carries a `Catalogue-Check` line in its References cell (one of `reuse|extend|no-match <org/repo>@<sha>`). For HRDs whose catalogue-check is still pending, the row records `no-match (2026-05-20)` — operators reviewing the implementation PR will re-verify.

ID	Type	Status	Criticality	Title	Opened
HRD-015	task	open	low	Inheritance gate I8 invariants for Go scaffold (go.work + commons/types.go + null adapter test passes)	2026-05-20
HRD-018	task	in_progress	high	`commons_constitution` Go package: Evaluator interface + 12 event-class emit helpers + constitution_state + constitution_bindings table migrations + bundle-hash captureer + mode-ladder runtime config	2026-05-20
HRD-019	task	open	high	`cherald` constitution bindings — bulk implementation: ~30 `.policy.violation` rules + `.gate.failed`/`.gate.recovered`/`.credential.leak`/`.sp` handlers; `/v1/compliance` pull surface	2026-05-20
HRD-020	task	open	high	`sherald` host-safety + repo-safety bindings (§9.1/.2/.3, §12.1/.2/.3/.6, §11.4.32, §11.4.36, §11.4.41, §11.4.71) — destructive-op detection hook, force-push interceptor, mem-budget watcher	2026-05-20

ID	Type	Status	Criticality	Title	Opened
HRD-021	task	open	middle	`bherald` CI/test bindings (§1, §11.4.2/.3/.4/.5/.7/.13/.14/.24/.27/.39/.43/.46/.48–.52/.67) — gate-result event emitters; integrates with the consuming project's CI workflow	2026-05-20
HRD-022	task	open	middle	`rherald` release bindings (§4 tag mirroring, §5 changelog, §11.4.38 installable-asset evidence, §11.4.40 full-suite retest)	2026-05-20
HRD-023	task	open	middle	`pherald` project bindings (§2 commit+push, §3 submodule propagation, §11.4.11/.15/.21/.22/.34/.37/.42/.55/.66/.71/.74)	2026-05-20
HRD-024	task	open	middle	`iherald` constitution-rule escalation bindings (§11.4.10/.10.A credential-leak page-out, §11.4.21 + §11.4.66 operator-blocked escalation)	2026-05-20
HRD-025	task	open	low	`scherald` scheduled-audit bindings (§11.4.45 periodic Status.md sweep + daily/weekly/monthly compliance digest)	2026-05-20
HRD-026	task	open	middle	Constitution-bundle hash captureer — computes SHA-256 of rendered Constitution.md at evaluation time; persists on every emitted event for replayability	2026-05-20
HRD-027	task	open	middle	Mode-ladder runtime config (`constitution_bindings` table + admin REST endpoints to flip allow/warn/enforce per binding per tenant without redeploy)	2026-05-20
HRD-029	task	open	middle	§2 `pherald commit-push` — single-entrypoint locked commit + multi-mirror push	2026-05-20
HRD-030	task	open	middle	§3 `pherald submodule propagate` — owned-submodule walk in propagation order	2026-05-20
HRD-031	task	open	middle	§4 `rherald tag mirror` — assert tag exists on every owned submodule	2026-05-20
HRD-032	task	open	low	§5 `rherald changelog generate` — Conventional Commits → `docs/changelogs/<v>.md` + multi-format export	2026-05-20
HRD-033	task	open	high	§9.1 `sherald destructive guard <op>` — wrap rm/git-reset/git-push-force with prerequisite checks	2026-05-20
HRD-034	task	open	middle	§9.3 `sherald backup snapshot <path>` — hardlinked snapshot helper	2026-05-20
HRD-035	task	open	middle	§11.4.2 `bherald evidence capture <test_id>` — captured-evidence recorder	2026-05-20
HRD-036	task	open	high	§11.4.10 `cherald creds scan` — gitleaks/trufflehog integration, emits `.credential.leak`	2026-05-20
HRD-037	task	open	low	§11.4.12 `cherald docs sync` — regen Issues_Summary / Fixed_Summary / Status_Summary	2026-05-20
HRD-038	task	open	low	§11.4.18 `cherald script-docs check` — assert sibling .md for every scripts/**/*.sh	2026-05-20
HRD-039	task	open	low	§11.4.19 / .23 `cherald fixed align` + `cherald colorize` — Issues/Fixed format + HTML colorizer	2026-05-20
HRD-040	task	open	high		2026-05-20

ID	Type	Status	Criticality	Title	Opened
				§11.4.26 `sherald constitution pull` — wrap fetch + rebase + validation gate, emits `.bundle.updated`	
HRD-041	task	open	middle	§11.4.27 `bherald test-tier verify` — 8-tier matrix verification	2026-05-20
HRD-042	task	open	low	§11.4.31 `cherald submanifest verify` — Submodule-Dependency-Manifest gate	2026-05-20
HRD-043	task	open	high	§11.4.36 `pherald install-upstreams` — install_upstreams wrapper	2026-05-20
HRD-044	task	open	middle	§11.4.37 `pherald fetch-guard` — pre-edit fetch + rebase enforcement	2026-05-20
HRD-045	task	open	high	§11.4.40 `rherald gate retest` — pre-tag full-suite retest gate	2026-05-20
HRD-046	task	open	high	§11.4.41 `sherald force-push gate` — merge-first + per-session-auth enforcement	2026-05-20
HRD-047	task	open	middle	§11.4.45 / .56 `scherald status digest` — periodic Status.md sweep + Status_Summary regen	2026-05-20
HRD-048	task	open	low	§11.4.53 `cherald fixed-summary sync` — standalone Fixed_Summary backfill	2026-05-20
HRD-049	task	open	middle	§11.4.55 `pherald reopen <HRD-NNN>` — Issues→Fixed reversal + Reopens history	2026-05-20
HRD-050	task	open	middle	§11.4.59 `cherald readme sync` — README doc-link regen + multi-format re-export	2026-05-20
HRD-051	task	open	high	§11.4.60 `cherald composite-gate` — canonical implementation of CM-DOCS-COMPOSITE-SYNC	2026-05-20
HRD-052	task	open	middle	§11.4.65 `cherald export` — bulk Markdown export wrapper (md/html/pdf/docx)	2026-05-20
HRD-053	task	open	high	§11.4.71 `pherald pre-push` — fetch + investigate + integrate hook	2026-05-20
HRD-054	task	open	middle	§11.4.73 `cherald spec-version check` — Revision-vs-edits drift detection	2026-05-20
HRD-055	task	open	middle	§11.4.74 `cherald catalogue-check <pr>` — scan PR for Catalogue-Check + survey runner over vasic-digital + HelixDevelopment	2026-05-20
HRD-056	task	open	high	§12.6 `sherald mem-budget watch` — daemon-mode 60% threshold watcher emitting `.host.safety_breach`	2026-05-20
HRD-081	task	open	low	Extend `digital.vasic.containers/pkg/compose` to detect podman-compose vs docker compose runtime and either emit runtime-appropriate flags (no `--wait` for podman-compose) or fall back to host-side healthcheck polling. Composes with §11.4.76 — required so `compose.WithWait(true)` works across both backends. Also fix `Status()` parsing for podman-compose ps output (returns 0 services even when containers are visible to podman directly). Per §11.4.76: extend upstream submodule, never reimplement.	2026-05-20

ID	Type	Status	Criticality	Title	Opened
HRD-085	task	open	middle	Implement single-task admin operations on `commons_infra.pgxTaskRepository` — `GetByID`, `Update`, `Delete`. Required for Requeue + MoveToDeadLetter call chains and operator tooling. Currently all three return `ErrUnsupported (HRD-085)`. Scope: read one BackgroundTask row by ID; full-row Update including JSONB columns; soft- or hard-Delete decision per §107.	2026-05-20
HRD-086	task	open	middle	Implement worker-side running-task state mutations on `commons_infra.pgxTaskRepository` — `UpdateStatus`, `UpdateProgress`, `UpdateHeartbeat`, `SaveCheckpoint`. Required by River workers + the stuck-detector reaper. Currently all four return `ErrUnsupported (HRD-086)`. Scope: status transitions guarded by valid-state-machine check; progress as `(float64, message)`; heartbeat as `updated_at = now()` writeback; checkpoint as opaque `[]byte` blob persisted alongside the task row.	2026-05-20
HRD-087	task	open	low	Implement admin / stats / audit reads on `commons_infra.pgxTaskRepository` — `GetByStatus`, `GetPendingTasks`, `CountByStatus`, `GetTaskHistory`. Drives the `/v1/queue/*` introspection endpoints + Peek delegate. Currently all four return `ErrUnsupported (HRD-087)`. Scope: status-filtered paginated reads; pending-task slice for Peek; status-bucketed counts; per-task execution-history rows from the `task_execution_history` table.	2026-05-20
HRD-088	task	open	middle	Implement queue-recovery reads on `commons_infra.pgxTaskRepository` — `GetStaleTasks`, `GetByWorkerID`. Drives the stuck-task reaper + worker-crash recovery loop. Currently both return `ErrUnsupported (HRD-088)`. Scope: stale-threshold filter against `updated_at`; per-worker assignment lookup for re-dispatch after a worker disappears.	2026-05-20
HRD-089	task	open	low	Implement resource-monitor I/O on `commons_infra.pgxTaskRepository` — `SaveResourceSnapshot`, `GetResourceSnapshots`. Persists per-task CPU/ memory/IO samples for capacity-planning + post-mortem. Currently both return `ErrUnsupported (HRD-089)`. Scope: append-only insert keyed by `(task_id, captured_at)`; per-task paginated read in reverse-chronological order. Schema target: a `task_resource_snapshots` table not yet in the §9.6 migration set — adding it is part of this HRD.	2026-05-20
HRD-090	task	open	middle	Implement dead-letter handling on `commons_infra.pgxTaskRepository` — `MoveToDeadLetter`. Required failure path when a	2026-05-20

ID	Type	Status	Criticality	Title	Opened
				task exhausts retries or fails a §107 invariant. Currently returns `ErrUnsupported (HRD-090)`. Scope: atomically move the BackgroundTask row to a `dead_letter_tasks` table with the failure-reason string + original-row JSONB snapshot; emit a `.queue.dead_letter` event via `commons_constitution` per §42.1. Schema target: a `dead_letter_tasks` table not yet in the §9.6 migration set — adding it is part of this HRD.	
HRD-131	task	open	middle	Migrate text HRD trackers to versioned SQLite single-source-of-truth (§11.4.93 / Herald §108.h, 6-phase) — DB committed as authoritative SSoT per operator mandate 2026-05-27. Phase 1 (this item): file the scope-locked tracking HRD. Phase 2: scaffold/adopt the migration Go binary that already lives in the constitution submodule (`constitution/scripts/workable-items/`) — Herald references it per §11.4.74 catalogue-first and never reimplements it. Phase 3: sync md→db (load `docs/Issues.md` / `docs/Fixed.md` / `*_Summary.md` / `docs/Status.md` / `docs/CONTINUATION.md` §3 into `docs/.workable_items.db`). Phase 4: sync db→md byte-identical (round-trip regeneration so MD?DB drift is mechanically impossible). Phase 5: generator shims (the `*_Summary.md` variants + Status/CONTINUATION become derived from the DB). Phase 6: prohibit text-direct edits (release-gate guard that the DB is the only authoring surface). Operator divergence from the parent §11.4.93 gitignored-with-regeneration default: for Herald the DB IS the authoritative artefact — `docs/.workable_items.db` is version-controlled (committed), NOT regenerated-on-clone; only the transient WAL/SHM sidecars (`docs/.workable_items.db-wal` / `-shm`) stay gitignored ("We should not git ignore our workable items database since it is our single source of truth regarding items we have to do on the project!"). **Scope-lock: this HRD covers Phase 1 (this row) + Phase 2 adoption ONLY.** Phases 3-6 (md→db sync, db→md byte-identical regeneration, generator shims, text-direct-edit prohibition) are follow-on work-items to be opened as Phase 2 completes, so the §106 spec-change / §107.x evidence invariants are never broken mid-migration.	2026-05-27
HRD-122	task	in_progress	middle	§11.4.85 stress+chaos shared scaffold — Go-native `commons/stresschaos` test-helper package (GAP-3 plan §2 / §6 T1). Provides: a bounded concurrent load-runner (`RunLoad` — N workers × M iterations of a closure via `sync.WaitGroup`, no new dependency: `golang.org/x/sync/errgroup` is NOT in go.mod so stdlib `sync` is used per the plan's "prefer NOT adding deps" tooling note); nearest-rank percentile	2026-05-27

ID	Type	Status	Criticality	Title	Opened
HRD-123	task	in_progress	high	math (`Percentiles` → `{p50,p95,p99,max,min,count}`) emitted as `latency.json` + `latency_histogram.csv`; an evidence-dir writer creating `docs/qa/<run-id>/stress_chaos/<surface>/`; and a best-effort host-memory headroom probe (`HostMemHeadroom` — `vm_stat`+`sysctl hw.memsize` on darwin, `/proc/meminfo` on linux) for the §12.6 60%-ceiling proof (degrades to `probe-unavailable`, never a hard failure). Self-tested: 8 unit tests (percentile correctness on a known 1..100ms dataset, evidence-dir creation, bounded-goroutine proof, host-mem best-effort, §12.6 ceiling predicate) PASS under `go test -race -count=1`. Lowest sensible layer per CLAUDE.md layering (commons L0, importable by every flavor). §11.4.85 Gin `/v1/events` stress+chaos tests (GAP-3 plan §1 row 1 / §6 T2) — new `pherald/internal/http/events_stress_chaos_test.go` driving the REAL POST `/v1/events` handler (`EventsHandler` → `runner.Run`, all 7 stages) over a REAL in-process `httptest` server dialed by `net/http.Client`, through the production Gin middleware chain (`cli.TOONMiddleware` + auth gate). Only the external boundary (PG/Redis/channel via `runner.NewFakeRunner`) is faked per §11.4.27; the handler, body read+transcode, error→status mapping, and orchestration run unmodified. STRESS: N=12 workers × M=200 = 2400 CloudEvent POSTs, `-race` clean — 0 errors, 0 5xx, all 202; p50=0.706ms/p95=1.755ms/p99=2.665ms/max=5.722ms, ~13,322 req/s. CHAOS: (b) duplicate-key-under-load (1200 POSTs, per-worker keys) → 12×202 + 1188×200, 0 5xx, 0 hang — graceful degrade; (c) input-corruption (7 bodies: 8 MiB random, truncated JSON, non-UTF8, JSON array, missing-id, garbage, empty) × 5 reps → every body a tagged `event_parser`: 400, 0 panic, 0 5xx; (d) auth storm — 1000 random 32-byte bearer tokens through the REAL `commons_auth.GinMiddleware`+HMAC(HS256) `Verifier` → 1000×401, 0 bypass (no token minted → no new `go.mod` dep); (a) live PG-drop (container pause) → SKIP-with-reason (no runtime; fail-loud-on-PG-error proven hermetically at runner layer HRD-125). Captured evidence under `docs/qa/HRD-123-stress-chaos-2026-05-27T132755/stress_chaos/events/` (latency.json, latency_histogram.csv, throughput.csv, redis_down_fallback.log, categorised_errors.txt, auth_storm.log, recovery_trace.log, host_memory_headroom.txt, summary.md). **FINDING (→ HRD-132 confirmed at HTTP layer): a SHARED-**	2026-05-27

ID	Type	Status	Criticality	Title	Opened
HRD-124	task	in_progress	high	**key cross-worker concurrent replay flood triggers the REAL** `runner.go:132 CachedRcpt.WasReplay` **data race** — the `NewFakeRunner` events_processed store caches+shares the `*Receipt`, so concurrent replays mutate it (this IS the "activates with Wave 4+ Receipt caching" case HRD-125 predicted). Test uses per-worker keys (production-faithful: real PG does NOT cache the Receipt today) so the HTTP-surface degrade contract is proven `-race`-clean without asserting a property the code cannot honour (= §107 PASS-bluff); the race is OWNED by HRD-132. Host-mem headroom recorded; HRD-123 is in-process (not a resource-exhaustion surface) so the §12.6 ceiling gate does not gate it. §11.4.85 Gin `/v1/compliance` (cherald) + `/v1/safety_state` (sherald) GET stress+chaos tests (GAP-3 plan §1 row 2 / §6 T3) — two new test files, `cherald/internal/compliance/compliance_stress_chaos_test.go` + `sherald/internal/safety/safety_stress_chaos_test.go`, each driving the REAL flavor handler (`compliance.Handler` / `safety.Handler`) over a REAL in-process `httptest` server dialed by `net/http.Client`, through the production Gin chain (`cli.TOONMiddleware` → auth → handler). Only the external boundary is faked per §11.4.27: `state.NewMemory()` (healthy) or `errStore{List→conn-error}` (PG-drop) for compliance; the auth claims-injector for both; NO production source edited, NO new `go.mod` dep (toon resolves via the existing `commons/cli` replace chain; the auth storm fires random bytes so no token is minted). STRESS (each flavor): N=10 workers × M=200 = 2000 GETs, `-race` clean — 0 errors, 0 5xx, all 200. Compliance p50=1.16ms/p95=2.00ms/p99=2.59ms ~8.4k req/s; safety_state p50=0.49ms/p95=1.43ms/p99=1.90ms ~16k req/s. **Content-negotiation correctness under concurrency** — every request alternates `Accept: application/toon` vs `application/json`; 1000 toon + 1000 json each flavor → 100% Content-Type match to the request's OWN Accept header (0 codec-bleed) AND every toon body is real TOON wire bytes (byte-0 ≠ `{`/`[`). CHAOS: (a) compliance PG-drop (`errStore` → store-down) → 2000/2000 fail-loud 500, EACH 5xx body naming the failed dependency (`store list failed`), 0 fabricated 2xx (a swallowed-error 200 `{results:[]}` would falsely tell an operator "no violations" while PG is down — the §107 mode this gate forbids); live container-pause variant SKIP-with-reason (no runtime; proven hermetically). (a-safety) safety_state has NO DB	2026-05-27

ID	Type	Status	Criticality	Title	Opened
				dependency on the hot path by design (Wave 3 §3 — in-memory `Aggregator` snapshot), so the PG-drop-as-5xx variant is honestly SKIP-not-applicable (asserting a non-existent dependency would be a reverse-§107 bluff); the code-true fault analogue — concurrent state-mutation under load (4 background goroutines hammering `UpdateMemPercent`+`RecordDestructiveOp` during 2000 reads) → all 200, 0 torn read, 0 5xx, `-race`-clean RWMutex proof — IS exercised instead. (b) auth storm (both flavors): 1000 random 32-byte bearer tokens through the REAL `commons_auth.GinMiddleware`+HMAC(HS256) `Verifier` → 1000×401, 0 bypass. Verified `go test -race -count=3` deterministically green across ALL 3 runs (no timing-sensitive assertion — the empty-Accept-defaults-to-TOON gotcha that made an early shape-check flaky was fixed by pinning `Accept: application/json` on the byte-inspection paths, per the HRD-123 flaky-oversized-body lesson). Captured evidence under `docs/qa/HRD-124-20260527T134316/stress_chaos/{compliance,safety_state}/` (latency.json, latency_histogram.csv, throughput.csv, pg_drop_fail_loud.log / concurrent_mutation.log, auth_storm.log, pg_drop_live.log / pg_drop_not_applicable.log, summary.md with §12.6 host-mem headroom probe). Bounded load only (§12/§12.6): small GETs, N≤10 workers, 4 bounded background goroutines, no fork/GB-alloc/host-net-change.	
HRD-125	task	in_progress	high	§11.4.85 Runner 7-stage pipeline stress+chaos tests (GAP-3 plan §1 row 3 / §6 T4) — new `pherald/internal/runner/stress_chaos_test.go` exercising the REAL `runner.Run` (only the external boundary — PG events_processed store, channel sink, Redis SETNX seam — is faked per §11.4.27; the orchestrator + 7 stages run unmodified). STRESS: N=16 goroutines × 40 iters concurrent same-key replay + fresh keys, `-race` clean (PASS). CHAOS: (a) 1000× duplicate flood @ 50 parallel under live atomic Redis SETNX; (a') same flood under the real nil-Redis PG-fallback degrade; (b) hermetic PG-deadlock (SQLSTATE 40P01) injection on the events_processed Insert. Captured evidence under `docs/qa/HRD-125-stress-chaos-2026-05-27T131251-gap3/stress_chaos/runner/` (latency.json p50=0.252ms/p95=1.075ms/p99=2.966ms, histogram CSV, race_clean.log, exactly_once.txt, duplicate_flood.txt, nil_redis_degrade.txt, deadlock_recovery.log, host_memory_headroom.txt, summary.md). **FINDING (→ HRD-132): dispatch**	2026-05-27

ID	Type	Status	Criticality	Title	Opened
HRD-126	task	in_progress	high	**exactly-once is NOT guaranteed under concurrent replay** — the events_processed archive row is written at Stage 7 (end of pipeline), so the Stage-2-fresh-verdict → Stage-7-archive-commit window admits a bounded handful of concurrent duplicate dispatches (observed 2–4 of a 1000× flood; never the full 1000). Archival (events_processed) IS exactly-once (PG UNIQUE + ON CONFLICT DO NOTHING). Tests assert the honest code-true contract (archival exactly-once + bounded dispatch), NOT an aspirational `sends==1` (that would be a §107 PASS-bluff). A latent `runner.go:132 CachedRcpt.WasReplay` data race surfaces only when a store caches+shares a `*Receipt` (current production PG Lookup returns Receipt=nil → no race today; would activate with the planned Wave 4+ Receipt caching). §11.4.85 `pherald listen` inbound-path stress+chaos tests (GAP-3 plan §1 row 4 / §6 T5) — two new test files: `pherald/internal/inbound/stress_chaos_test.go` (in-package `inbound`, so it can exercise the UNEXPORTED `extractReplyToMessageID`) + `pherald/cmd/pherald/listen_stress_chaos_test.go` (drives the REAL `runListen` loop seam). Per §11.4.27 only the channel boundary (Subscriber) + CC boundary (`CodeDispatcher`) are faked; the `inbound.Dispatcher`, its action-routing switch, the `runListen` fan-in orchestration + clean-shutdown path run unmodified. STRESS: (1) N=16 workers × 50 = 800 concurrent `Dispatcher.Handle` calls — 0 errors, exactly-once reply per event (800/800), goroutine 2→2 (leaked=0), p50=0.41ms/p95=1.64ms/p99=2.21ms/max=3.12ms ~25k dispatch/s; (2) 400-event burst through the full `runListen` loop — nothing dropped, clean shutdown on cancel (<3s, SIGTERM analogue), no leak, ~18k events/s. CHAOS: (a) table-driven corrupt `Raw` payloads (nil map, missing/nil/string/bool/slice/map `message_id`) → `extractReplyToMessageID` degrades to (0,err); valid int/int64/int32/float64/float32 decode; under a 64-dispatch corrupt-Raw flood `Dispatcher.Handle` is panic-free (recover-guard), 0 errors, degraded `replyTo=""` (no bogus quote id) — HERMETIC CORE; (b) CC subprocess fault — 6 cases (subprocess-killed `errors.Is`-reachable / truncated / marker-less / empty / unknown-action) → stage-tagged `inbound:` error, NO reply leak, no hang (3s watchdog); (c) once-only side-effect — 50 workers × 20 = 1000 redeliveries of the same `issue.open` collapse to exactly-ONE durable sink row (every redelivery reaches the sink: attempts==1000) — the in-process analogue of a crash+restart redelivery loop, leaning on the once-only contract HRD-125 proved at the Runner	2026-05-27

ID	Type	Status	Criticality	Title	Opened
				layer; the real-binary SIGKILL+restart variant is SKIP-with-reason (`HERALD_STRESS_LIVE_LISTEN=1` needed: operator Telegram creds + live PG + built pherald). Verified `go test -race -count=3` deterministically green across ALL 3 runs on both packages (assertions are value-true: exactly-once reply count, exactly-one durable row, tagged-error substrings — not timing-sensitive). NO production source edited, NO new `go.mod` dep (`commons/stresschaos` resolves via the existing `commons` replace chain). Captured evidence under `docs/qa/2026-05-27T135301-gap3-listen-HRD-126/stress_chaos/listen/` (latency.json, latency_histogram.csv, throughput.txt, listen_loop_throughput.txt, malformed_payloads.txt, handle_malformed_raw.txt, cc_fault.txt, once_only_side_effect.txt, channel_death.txt, process_death_restart.log, hostmem.txt, summary.md). Bounded load only (§12/§12.6): N≤50 goroutines, KiB-scale alloc, in-process `context.CancelFunc` only — no host/system process signalled, no fork/GB-alloc/host-net-change.	
HRD-127	task	in_progress	high	§11.4.85 claude_code dispatch stress+chaos tests (GAP-3 plan §1 row 5 / §6 T6) — new `commons_messaging/dispatch/claude_code/dispatch_stress_chaos_test.go` (in-package `claude_code`, so it exercises the UNEXPORTED `buildCmd`/`bootstrapSession`/`parseReply`) + 5 committed hermetic shims under `commons_messaging/dispatch/claude_code/testdata/fake-claude-*.sh`. Per §11.4.27 only the EXTERNAL boundary — the `claude` CLI binary — is faked, via those committed shims selected through the EXISTING `New(binaryPath,...)` constructor seam (the same seam `bootstrap_test.go`'s `writeFakeClaudeBinary` uses); NO production source edited, NO production env var added, NO new `go.mod` dep (`commons/stresschaos` resolves via the existing `commons` replace chain), NO real `claude` invoked. STRESS: (1) N=24 workers × 25 = 600 concurrent `Dispatch` calls (steady-state `--resume` path, anchor pre-seeded) — 0 errors, 0 bad-replies, every reply REALLY parsed from the shim's `<<<HERALD-REPLY>>>` line, every call resumed the SAME pre-seeded UUID (no double-bootstrap), anchor never drifted; p50=62.9ms/p95=96.4ms/p99=116.9ms/max=147.6ms ~363 dispatch/s (latency dominated by the per-call shell-shim `fork/exec` — honest exec round-trip cost); (2) N=16 concurrent COLD-START bootstrap (no anchor) — all dispatches succeed, exactly one	2026-05-27

ID	Type	Status	Criticality	Title	Opened

anchor UUID persists, `-race` clean. CHAOS: (a) process-death — `exit 137` (128+SIGKILL, partial stdout) → tagged `claude_code: dispatch claude --resume <u>: exit 137: killed mid-write` error, no hang (10s watchdog), partial stdout NOT accepted; + self-`kill -KILL $$` → tagged error, no hang; (b) timeout — fake `claude` sleeps 30s, ctx deadline 800ms → `exec.CommandContext` cancellation fires, Dispatch returns in 803ms (NOT 30s), tagged error; + bootstrap-timeout variant (600ms budget → returns 603ms, anchor NOT persisted on failure); (c) truncated `<<<HERALD-REPLY>>>` (half-JSON, exit 0) → `parseReply` returns explicit `decode reply JSON` error, NEVER a silent partial-accept; UNIT corruption taxonomy (truncated_json / half_marker / marker_no_brace / missing_closing_brace / empty) each → explicit error, well-formed reply still parses (discriminating, not blanket-reject). Verified `go test -race -count=3` deterministically green across ALL 3 runs (`DATA_RACE_lines=0`; assertions are value-true: exact parsed-reply fields, tagged-error substrings, bounded-time returns — not timing-sensitive). Captured evidence under `docs/qa/HRD-127-stress-chaos-2026-05-27T090518/stress_chaos/claude_code/` (latency.json, latency_histogram.csv, dispatch_latency.txt, cold_start_bootstrap.txt, subprocess_kill.log, subprocess_self_sigkill.log, timeout_cancel.log, bootstrap_timeout_cancel.log, truncated_reply.txt, race_clean.log, host_memory_headroom.txt, summary.md). **FINDING-1: cold-start concurrent bootstrap is NOT exactly-once by design** (`bootstrap.go:50-52` — last-writer-wins, not serialised, first orphan inert); asserting `bootstrap_count==1` would be a §107 PASS-bluff, honest bound (bootstraps∈[1,workers]) asserted instead; fix direction if ever required = advisory flock on the anchor. **FINDING-2: timeout-cancellation latency depends on `claude` being a single process** — production `Dispatch` wires ctx into `exec.CommandContext` whose default Cancel SIGKILLs only the DIRECT child, and `cmd.Output()` blocks until the stdout pipe is closed by ALL descendants; the real `claude` is a single long-lived process so on SIGKILL it closes its own stdout and Output() unblocks immediately (verified 803ms). The fake sleep shim uses `exec sleep` (single PID, replace-in-place) to model this faithfully; if `claude` ever spawns helper subprocesses that outlive a SIGKILL while holding stdout, cancellation latency would degrade — hardening direction = `Cmd.Cancel` + process-group kill (Setpgid + kill(-pgid)); flagged for the dispatcher

ID	Type	Status	Criticality	Title	Opened
HRD-128	task	in_progress	high	owner, out of scope for the HRD-127 test layer. Bounded load only (§12/§12.6): N≤24 goroutines, each spawning ONE short-lived fake-shim child the test owns+reaps; the ONLY process ever killed is a fake-shim child this test spawned (self-SIGKILL or ctx-driven kill) — never a real `claude` or host process; no fork/GB-alloc/host-net change. claude_code is an in-process `exec` surface (not a resource-exhaustion surface) so the §12.6 60%-memory ceiling does not gate it (same posture as HRD-123/HRD-126); host-mem probe recorded for the audit trail. §11.4.85 container-orchestrated / resource-exhaustion stress+chaos tests (GAP-3 plan §1 row 6 / §6 T7 + §7 host-safety) — new `commons_storage/resource_stress_chaos_test.go` (in-package `storage`, hermetic core) + `tests/test_resource_stress_chaos.sh` (LIVE container harness). NO production source edited, NO new `go.mod` dep (`commons/stresschaos` resolves via the existing `commons → ../commons` replace in `commons_storage/go.mod`; only the `digital.vasic.database` boundary is faked per §11.4.27). **HERMETIC PASS (×2, deterministic):** (1) **disk-full error propagation** — a fake `database.Database`/`Tx` injects `syscall.ENOSPC` ("no space left on device") on the migration-UP exec, driving the REAL `RunMigrations → applyMigration` write path (runner-bookkeeping DDL/INSERT succeed so the fault lands on a real migration body); asserts `up_exec_attempts>=1` (real write path exercised), error returned (NO silent swallow), error tagged `commons_storage: apply` + `exec up`, `errors.Is(err, syscall.ENOSPC)` true (wrap chain unbroken, machine-classifiable), 0 migrations reported applied (no partial-success bluff). Captured full error: `commons_storage: apply v1 (init_core): exec up: write tablespace pg_default: no space left on device`. (2) **§12.6 host-mem headroom proof** — `HostMemHeadroom()` captured pre/post a 25× in-process disk-full workload; the workload's own `used_fraction_delta` is ≈0 (≈ -0.0003 — host needle did not move), proving the hermetic unit adds negligible host pressure; this is the §12.6 compliance-evidence artefact. **OPT-IN (deterministic SKIP without env):** real bounded-scratch-dir disk fill, HARD-capped at 64 MiB inside `os.MkdirTemp` with always-run defer-cleanup, gated behind `HERALD_STRESS_LIVE_DISK=1` (verified runnable + cleanup-clean when opted in; SKIPs deterministically by default). **LIVE container scenarios — SKIP-with-	2026-05-27

ID	Type	Status	Criticality	Title	Opened
				reason (§11.4.3, host-safety-gated): the shell harness owns (a) container OOM-confinement (`--memory=<cap>` container OOM-killed → confined to container cgroup scope, host stays <60% per §12.6) and (b) ≥30s connection-churn against a `--memory`-bounded PG; BOTH gated behind `HERALD_STRESS_LIVE_OOM=1` + a §12.6 pre-flight host-headroom gate that REFUSES to add memory pressure when host used-fraction ≥ 60% (or host_used + cap would cross 60%). On this run the host was at ~74% (pre-existing unrelated load), so the gate correctly REFUSED even when opted in — an honest SKIP over an unsafe run; podman present but never booted, no `herald-oom-*` container leaked. Verified `go test -race -count=3` deterministically green ALL 3 runs (PASS×3 / PASS×3 / SKIP×3) + shell harness ×3 deterministic (exit 0, 2 SKIPs, 1 preflight evidence each); `go vet` clean; full `commons_storage` package regression-clean. §12 / §12.6 host-safety: NO host-OOM, NO real-host-disk fill, NO host/system process killed, NO `systemctl`, NO host-network change, NO GB-alloc — the whole point of this unit honoured. Captured evidence under `docs/qa/HRD-128-resource-2026-05-27T091641/stress_chaos/resource/` (disk_full_tagged_error.txt, host_memory_headroom.txt [§12.6 proof], host_memory_preflight.txt, container_oom_confinement.skip.txt, connection_churn_stress.skip.txt, summary.md).	
HRD-132	bug	open	high	Tighten Runner idempotency window so concurrent same-key replay cannot duplicate-dispatch a notification (FINDING from HRD-125 stress/chaos). `idempotency.go:64-72` treats a Redis-SETNX-miss-with-no-PG-row as FRESH (the documented §32.1 "Redis-lies-PG-truths" design, to avoid ingest lockup), but the `events_processed` archive row is only written at Stage 7 (`OutcomeRecorder.Process`) — so concurrent same-key requests arriving in the Stage-2→Stage-7 window ALL dispatch (observed 2–4 of a 1000× flood; never the full 1000). Archival is exactly-once (PG UNIQUE + `ON CONFLICT DO NOTHING`); channel *dispatch* is only bounded → for a notification system this is duplicate-alert delivery under concurrent load. Fix direction: claim the idempotency slot atomically BEFORE dispatch (`INSERT events_processed … ON CONFLICT DO NOTHING` at Stage 2, treat 0-rows-affected as duplicate, so the PG row — not Stage-7 — is the dispatch gate), preserving the nil-Redis PG-only degrade (HRD-179). ALSO fix the latent `runner.go:132` `CachedRcpt.WasReplay` data race (mutates a possibly-shared `*Receipt`; dormant today since PG	2026-05-27

ID	Type	Status	Criticality	Title	Opened
				Lookup returns Receipt=nil, activates with Wave 4+ Receipt caching). MUST ship with the §11.4.85 stress test upgraded to assert `sends==1` under the same 1000× flood (the stronger assertion HRD-125 deliberately could NOT make without this fix — §11.4.4 test-interrupt-on-discovery).	

In progress

ID	Type	Status	Criticality	Title	Opened	Last update	References
HRD-008	task	in_progress	middle	Operator-side quickstart compose validation (Postgres + Redis + OTel + pherald container)	2026-05-20	2026-05-20	spec V3 §26.5 — scaffold shipped, live end-to-end run pending operator; Catalogue-Check: no-match (2026-05-20) — bespoke to Herald

Blocked

(none)

Conventions

See `Fixed.md` for closed items + Universal §11.4.12/.15/.16/.19/.33/.55/.73/.74 composition rules.